Module 12 - chapter 6

Terraform commands

Difference between current state (state file) and desired state (tf config file)?

And I don't want to -> terraform apply. I only want to see the difference:

-> terraform plan

```
➜  terraform git:(Module_12/Terraform_chapter-6) terraform plan
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development-vpc: Refreshing state... [id=vpc-0e43e232c4a94ae2a]
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc90fa9]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # aws_subnet.dev-subnet-2 will be created
  + resource "aws_subnet" "dev-subnet-2" {
      + arn                                            = (known after apply)
      + assign_ipv6_address_on_creation                = false
      + availability_zone                              = "eu-west-2a"
      + availability_zone_id                           = (known after apply)
      + cidr_block                                     = "172.31.48.0/20"
      + enable_dns64                                   = false
      + enable_resource_name_dns_a_record_on_launch    = false
      + enable_resource_name_dns_aaaa_record_on_launch = false
```

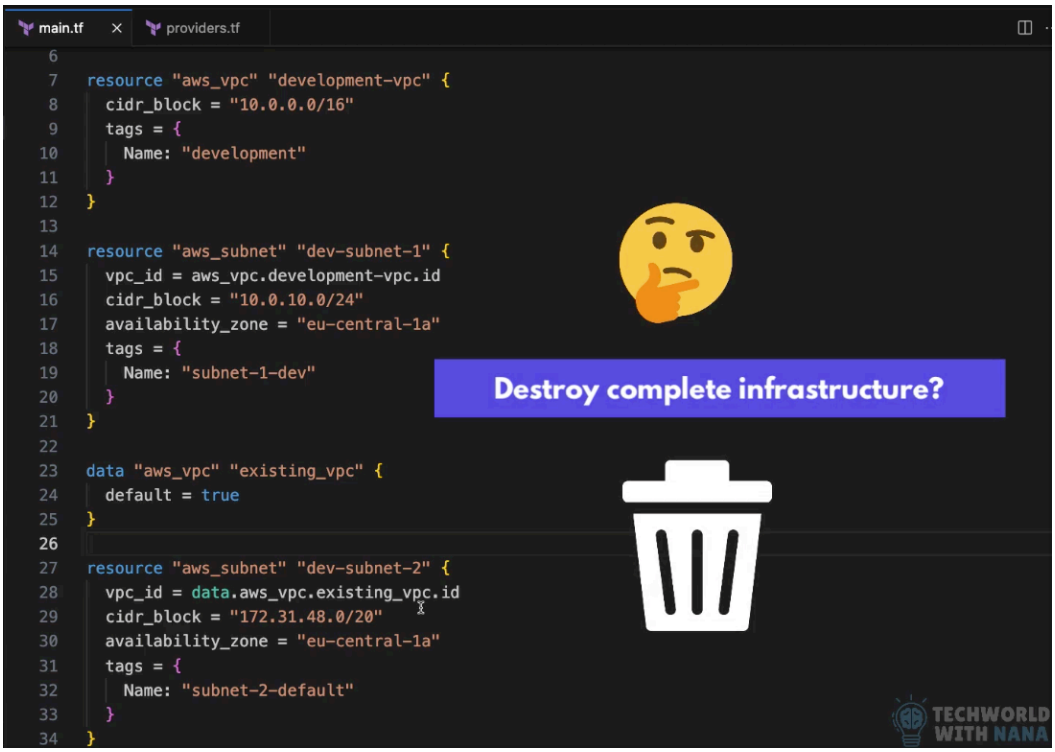
```
      + id                                         = (known after apply)
      + ipv6_cidr_block                            = (known after apply)
      + ipv6_cidr_block_association_id             = (known after apply)
      + ipv6_native                                = false
      + map_public_ip_on_launch                    = false
      + owner_id                                   = (known after apply)
      + private_dns_hostname_type_on_launch        = (known after apply)
      + region                                     = "eu-west-2"
      + tags                                       = {
          + "Name" = "subnet-2-default"
        }
      + tags_all                                   = {
          + "Name" = "subnet-2-default"
        }
      + vpc_id                                     = "vpc-03677c88f7cc2f715"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.
➜  terraform git:(Module_12/Terraform_chapter-6)
```

If we want to execute terraform apply and get approved without asking us to type ‘yes’ then
-> terraform apply -auto-approve

```
      + region                                     = "eu-west-2"
      + tags                                       = {
          + "Name" = "subnet-2-default"
        }
      + tags_all                                   = {
          + "Name" = "subnet-2-default"
        }
      + vpc_id                                     = "vpc-03677c88f7cc2f715"
    }

Plan: 1 to add, 0 to change, 0 to destroy.
aws_subnet.dev-subnet-2: Creating...
aws_subnet.dev-subnet-2: Creation complete after 0s [id=subnet-0ec05fa4170e48577]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
➜  terraform git:(Module_12/Terraform_chapter-6)
```

One Resource was created without user inputed ‘yes’

What fi I want to completely destroy my infra.



-> terraform destroy

And tf will destroy each element one by one -> It builds a dependency graph and destroys resources in the correct order based on dependencies.

For example if you have:
- A subnet that depends on a VPC
- An EC2 instance that depends on the subnet

Terraform will destroy in reverse dependency order:
1. EC2 instance first
2. Subnet second
3. VPC last

Because you can't delete a VPC while a subnet still exists inside it. Terraform figures this out automatically from the dependency graph.

```
      - id                                   = "vpc-0e43e232c4a94ae2a" -> null
      - instance_tenancy                     = "default" -> null
      - ipv6_netmask_length                  = 0 -> null
      - main_route_table_id                  = "rtb-0eddcdfd46e31eda6" -> null
      - owner_id                             = "706976333940" -> null
      - region                               = "eu-west-2" -> null
      - tags                                 = {
          - "Name" = "development"
        } -> null
      - tags_all                             = {
          - "Name" = "development"
        } -> null
        # (4 unchanged attributes hidden)
    }

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value:
```

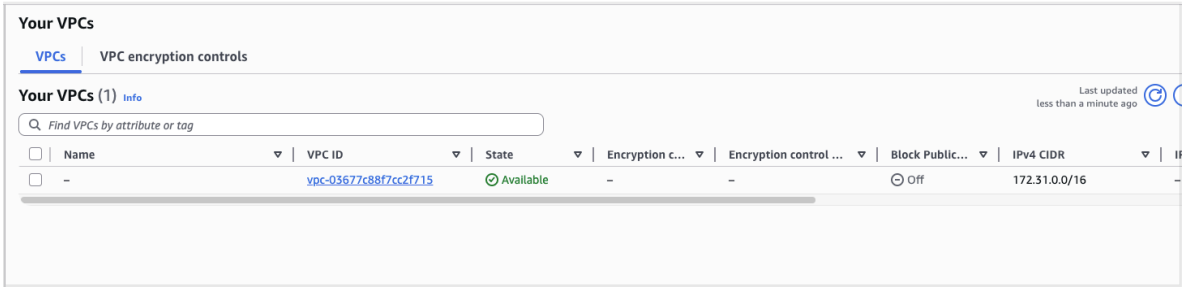
I confirm -> yes

```
Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_subnet.dev-subnet-2: Destroying... [id=subnet-0ec05fa4170e48577]
aws_subnet.dev-subnet-1: Destroying... [id=subnet-0ed81ef156fc90fa9]
aws_subnet.dev-subnet-1: Destruction complete after 1s
aws_vpc.development-vpc: Destroying... [id=vpc-0e43e232c4a94ae2a]
aws_subnet.dev-subnet-2: Destruction complete after 1s
aws_vpc.development-vpc: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.
➜  terraform git:(Module_12/Terraform_chapter-6)
```

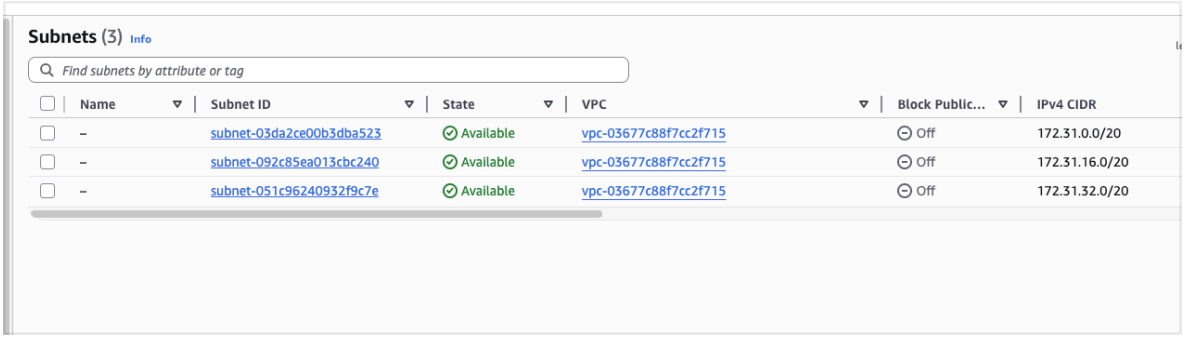
```
lan: 0 to add, 0 to change, 3 to destroy.

o you really want to destroy all resources?
 Terraform will destroy all your managed infrastructure, as shown above.
 There is no undo. Only 'yes' will be accepted to confirm.

 Enter a value: yes
```

You don't have to know which **order** you need to **delete the resources in**

Now we check AWS UI and the vPC is gone:



And the subnet is gone as well



I removed the credentials of my AWS account from my tf file instead I am setting the credentials as env vars per session (per terminal session) see notes ‘Shared AWS 'admin' acces key and secret via github’

Questions

Q:
 if I am working in a team, I guess I will use git and to know the most recent state of tf I will look into the main branch?

- Yes, but there's a bigger concern — the state file.

  If you're working in a team and everyone runs terraform apply locally, each person has their own local state file and they'll get out of sync, which causes big problems.

  The standard solution is remote state — storing the state file in a shared location like:
  - S3 bucket (most common for AWS)

- Terraform Cloud

That way everyone on the team is working against the same state file. You configure it in your terraform {} block:

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "state/terraform.tfstate"
    region = "eu-west-2"
  }
}
```

This is considered essential for team Terraform workflows.

Q:
so I will have a local branch and I will be doing my changes to tf then I push for a PR then it gets merge to main and main will use a remote state?

● Exactly. The typical team workflow is:

1. You work on your local branch, make changes to .tf files
2. Push and open a PR
3. A CI/CD pipeline (like GitHub Actions) runs terraform plan on the PR so the team can review what changes will happen in AWS
4. PR gets merged to main
5. CI/CD runs terraform apply on main against the remote state file

That way no one runs terraform apply locally — it always goes through the pipeline, using the shared remote state. This avoids conflicts and keeps everything in sync.

Q:
but when I am working in my branch I will get the remote state, I will update my tf but I won't run terraform apply? or can I run terraform apply in a dev env?

● You have two common approaches:

Option 1 — No local apply
You only edit .tf files locally, run terraform plan to preview changes, but never terraform apply. The actual apply only happens via CI/CD on merge to main.

Option 2 — Separate dev environment (most common in teams)
Each developer gets their own dev AWS environment with its own remote state. So you can freely run terraform apply in dev without affecting others. Then when merged to main it applies to staging/prod.

The environments would have separate state files:
state/dev/terraform.tfstate
state/staging/terraform.tfstate
state/prod/terraform.tfstate

Option 2 is safer and more practical — it lets you test your changes for real before they hit production.